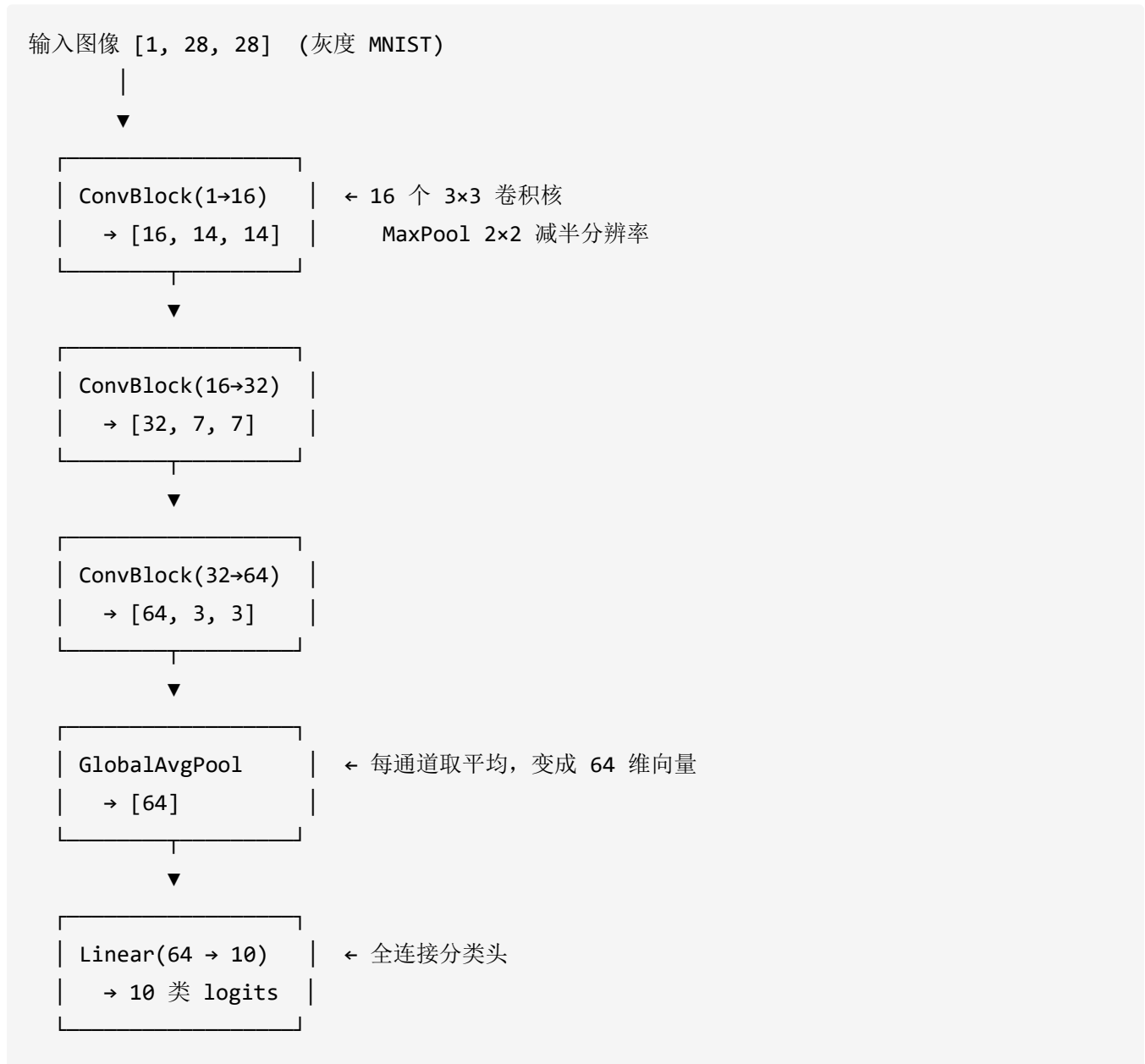




CNN 最简 Demo 解读

1. 整体架构

CNN（卷积神经网络）是计算机视觉的基石架构。核心思想是用**可学习的卷积核**在图像上滑动，逐层提取从低级到高级的特征。



2. 核心组件逐行解读

2.1 卷积层 (Conv2d)

```
self.conv = nn.Conv2d(in_ch, out_ch, kernel, padding=kernel // 2)
```

参数	含义	本例取值
in_ch	输入通道数	1 (灰度图)
out_ch	输出通道数 (= 卷积核数量)	16 → 32 → 64
kernel	卷积核尺寸	3×3
padding	边缘填充	1 (保持空间尺寸不变)

卷积运算本质： 一个 3×3 的权重矩阵在图像上滑动，每个位置做逐元素相乘再求和，产生一张特征图 (feature map)。
out_ch 个卷积核产生 out_ch 张特征图。

输入 5×5, kernel 3×3, padding=1 → 输出 5×5

[a b c d e]

[f g h i j]

[k l m n o]

[p q r s t]

[u v w x y]

卷积核

[w1 w2 w3]

[w4 w5 w6]

[w7 w8 w9]

⊗

= 特征图

2.2 批归一化 (BatchNorm2d)

```
self.bn = nn.BatchNorm2d(out_ch)
```

- 对每个通道，在 batch 维度上做归一化 (减均值除以标准差)
- 作用：** 稳定训练、允许更大学习率、减少对初始化的敏感性
- 在推理时使用训练期间累积的移动平均值

2.3 池化层 (MaxPool2d)

```
self.pool = nn.MaxPool2d(2) # 2x2 窗口取最大值
```

输入 4x4, pool=2 → 输出 2x2

[1 3 2 0]

[2 6 4 8]

[5 9 1 7]

[3 2 6 4]

→ MaxPool →

[6 8]

[9 7]

- **降维：** 将 $28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7 \rightarrow 3 \times 3$
- **平移不变性：** 轻微位移不影响最大值
- **减少计算量：** 每层空间尺寸减半

2.4 全局平均池化 (GlobalAvgPool)

```
self.pool = nn.AdaptiveAvgPool2d(1) # 输出固定为 1x1
```

- 每个通道的所有空间位置取平均，得到一个标量
- $[B, 64, 3, 3] \rightarrow [B, 64, 1, 1] \rightarrow \text{squeeze} \rightarrow [B, 64]$
- **替代 Flatten + 大 FC 层**，大幅减少参数量，防止过拟合

2.5 ConvBlock 组合

```
def forward(self, x):  
    return self.pool(F.relu(self.bn(self.conv(x))))
```

每个 ConvBlock 执行：**卷积** → **归一化** → **激活** → **池化**。这是 CNN 的标准构建块，三层堆叠逐步提取越来越抽象的特征：

层	感受野	学到的特征
Block1	小 (5x5)	边缘、角点、纹理
Block2	中 (14x14)	形状、笔画

层	感受野	学到的特征
Block3	大 (30×30)	数字整体结构

3. 特征图维度变化

以 batch_size=128 为例：

输入：	[128, 1, 28, 28]	
Conv1(1→16)：	[128, 16, 28, 28]	← 通道 1→16, padding=1 保持尺寸
MaxPool(2)：	[128, 16, 14, 14]	← 尺寸减半
Conv2(16→32)：	[128, 32, 14, 14]	
MaxPool(2)：	[128, 32, 7, 7]	← 再减半
Conv3(32→64)：	[128, 64, 7, 7]	
MaxPool(2)：	[128, 64, 3, 3]	← 再减半
GlobalAvgPool：	[128, 64, 1, 1]	
Squeeze：	[128, 64]	
Linear：	[128, 10]	← 10 类 logits

4. CNN vs 全连接网络

用 MNIST 举例，如果第一层就用全连接：

	全连接	CNN
输入	28×28=784 个神经元	[1, 28, 28]
第一层参数量	784×128 ≈ 100K	16×1×3×3 = 144
平移不变性	无（位置变了权重不认识）	有（卷积核到处滑动）

	全连接	CNN
空间结构	丢失（图像被展平）	保留

CNN 的两个关键设计：

- 1. **局部连接**：每个神经元只看 3×3 邻域，不看全图
- 2. **权重共享**：同一个卷积核在整个图像上复用，参数量与图像尺寸无关

5. 运行方式

```
pip install torch torchvision
python cnn_demo.py
```

预期输出：

```
Device: cpu
Train samples: 60000, Test samples: 10000
Parameters: 24,170
Epoch 1 | Train Loss: 0.4646  Acc: 91.16% | Test Loss: 0.1976  Acc: 94.41%
Epoch 2 | Train Loss: 0.0873  Acc: 97.95% | Test Loss: 0.1082  Acc: 96.87%
Epoch 3 | Train Loss: 0.0578  Acc: 98.49% | Test Loss: 0.0624  Acc: 98.30%
Epoch 4 | Train Loss: 0.0441  Acc: 98.86% | Test Loss: 0.0617  Acc: 98.20%
Epoch 5 | Train Loss: 0.0369  Acc: 99.03% | Test Loss: 0.0573  Acc: 98.18%

Sample predictions: [7, 2, 1, 0, 4, 1, 4, 9]
Ground truth:      [7, 2, 1, 0, 4, 1, 4, 9]
```

仅 ~2.4 万参数、5 个 epoch 就能在 MNIST 上达到 99% 准确率。

6. 关键设计决策

决策	原因
<code>kernel=3, padding=1</code>	保持空间尺寸不变，方便堆叠深层网络
<code>MaxPool(2)</code> vs <code>Conv stride=2</code>	MaxPool 更简单直观；现代网络倾向用 stride=2 卷积替代
<code>BatchNorm</code> 放在激活前	原始论文放在激活前；实践中 pre-activation（激活前）和 post-activation 各有优劣
<code>AdaptiveAvgPool</code> 而不是 Flatten	自适应任何输入尺寸，参数量固定

7. 从 Demo 到现代 CNN

这个 demo 可视为 **VGG 风格的简化版**（3×3 卷积 + MaxPool 堆叠）。现代 CNN 的演进：

- **ResNet (2015)**：引入残差连接（skip connection），允许训练 100+ 层网络
- **Inception (2014-2016)**：同一层内并行多个不同尺寸的卷积核
- **MobileNet (2017)**：深度可分离卷积（depthwise + pointwise），大幅减少计算量
- **EfficientNet (2019)**：用 NAS 搜索深度/宽度/分辨率的最优配比
- **ConvNeXt (2022)**：借鉴 ViT 的设计"现代化"CNN，缩小与 Transformer 的差距

但**核心的卷积-归一化-激活-池化流水线与本 demo 完全一致。**